

2026 edition

Deep Learning for Music Analysis and Generation

Music Foundation Models



Yi-Hsuan Yang Ph.D.
yhyangtw@ntu.edu.tw

Music Foundation Models

<https://github.com/nicolaus625/FM4Music>

- “The term foundation model was coined to describe a multipurpose machine learning model that, **rather than being trained for a single specific task**, serves as the **basis** of multiple derived models that are able to perform a wide range of tasks”
- “Following this **pre-training** phase, foundation models can be **adapted** for various **downstream** tasks via a relatively **lightweight finetuning** or **in-context learning** stage, for example, using a labelled dataset that is orders of magnitude smaller than the pre-training data.”
- “FMs for music not only address data scarcity and reduce annotation costs, but also enhance **generalisation** in music information retrieval and creation.”

Copyright-free Music Datasets; Good for Pre-Training

https://music-classification.github.io/tutorial/part2_basics/dataset.html

- Free Music Archive (FMA)
 - <https://freemusicarchive.org/>
 - <https://github.com/mdeff/fma>
- Jamendo dataset
 - <https://www.jamendo.com/>
 - <https://github.com/MTG/mtg-jamendo-dataset>
- FreeSound
 - <https://freesound.org/>
 - <https://labs.freesound.org/datasets/>

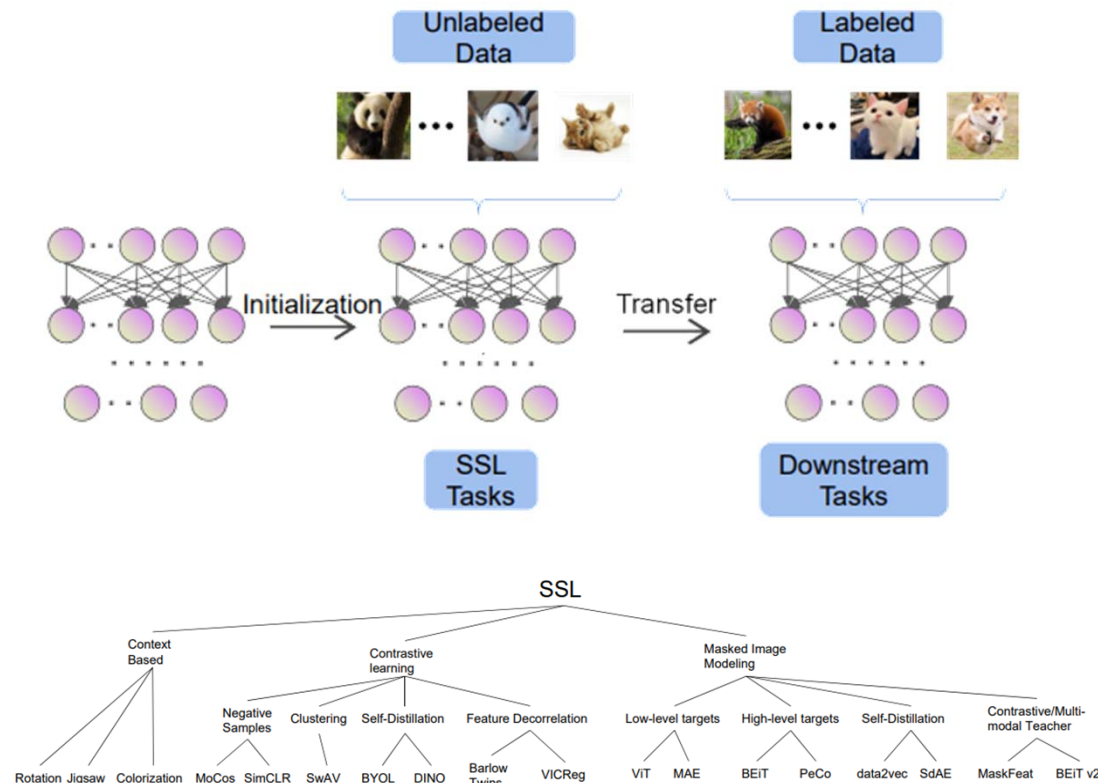
Pre-Training Strategies

- “Foundation models are pre-trained in a **self-supervised** fashion on **large-scale** datasets, **avoiding or minimising the need for labelled data**”
- Mainstream strategies
 - Generative pre-training (e.g., AE or VQVAE)
 - Contrastive learning (e.g., SimCLR)
 - Masked modelling (e.g., BERT or MAE)
 - Predictive joint-embedding (e.g., JEPA)

Self-Supervised Learning (SSL)

https://music-classification.github.io/tutorial/part5_beyond/introduction.html

- **Do not need** human labels (i.e., unlabeled)
- Can learn from **large** data
- Can do “**transfer learning**”
 - **Pre-train** the model with SSL
 - **Fine-tune** with **a few more layers** (usually linear layers) on downstream tasks using supervised learning

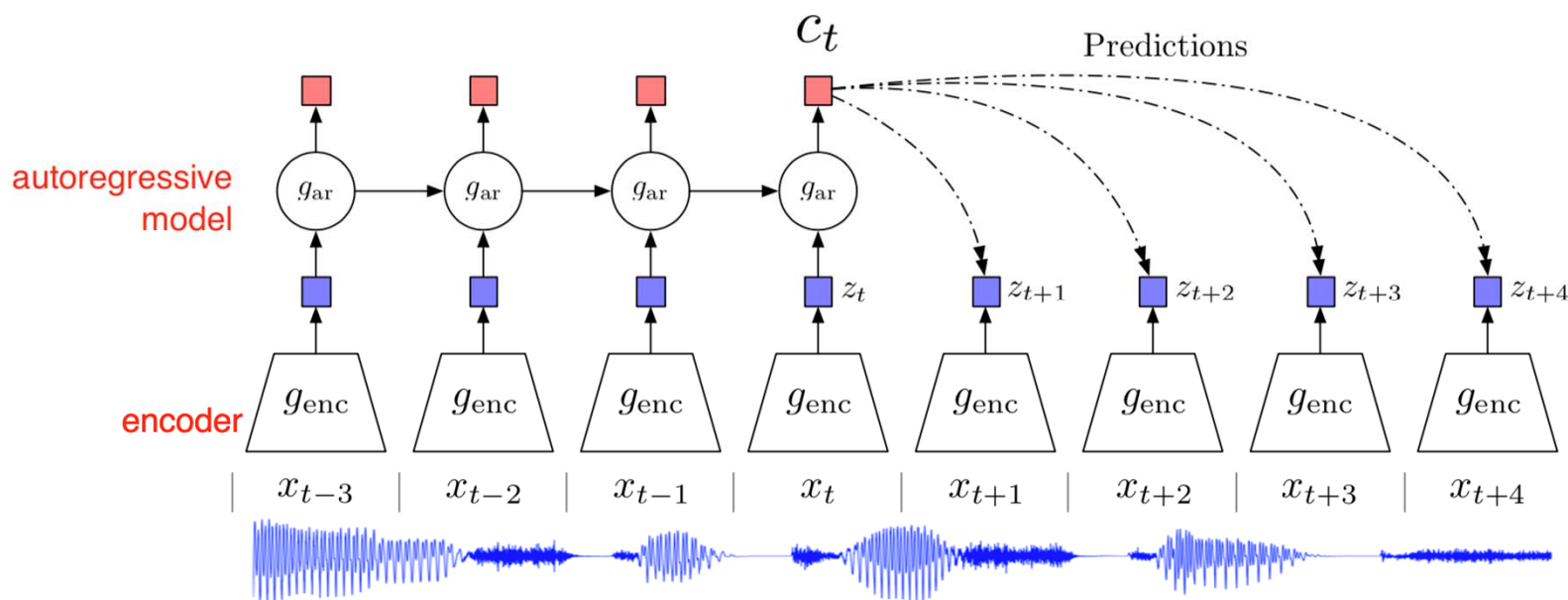


Gui et al., “A survey on self-supervised learning: Algorithms, applications, and future trends,” TPAMI 2024

Generative Pre-training: Contrastive Predictive Coding

https://music-classification.github.io/tutorial/part5_beyond/methods.html

- Learn by predicting the future in a learned latent space

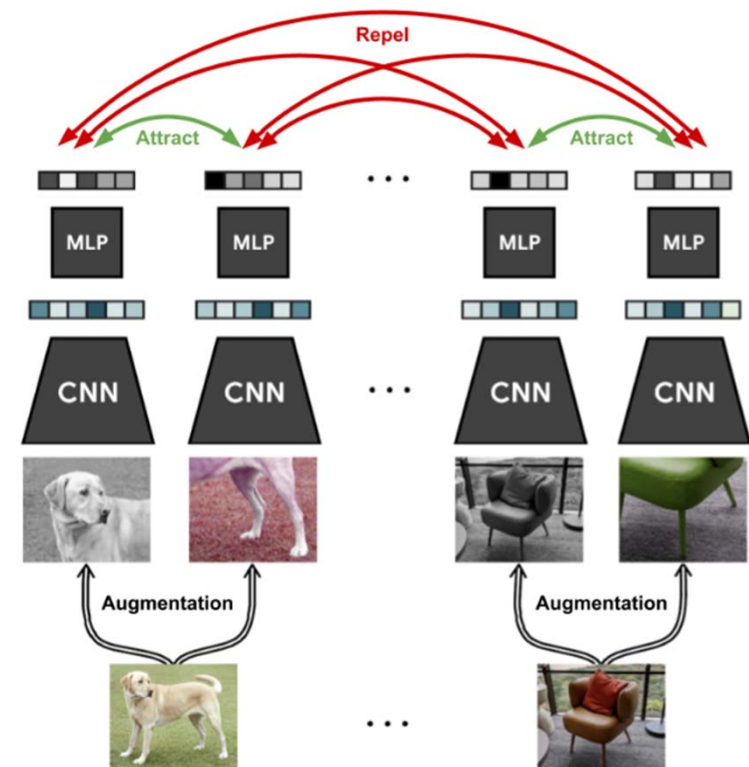


Ref: van den Oord et al., "Representation learning with contrastive predictive coding," arXiv 2018

Contrastive Learning: SimCLR

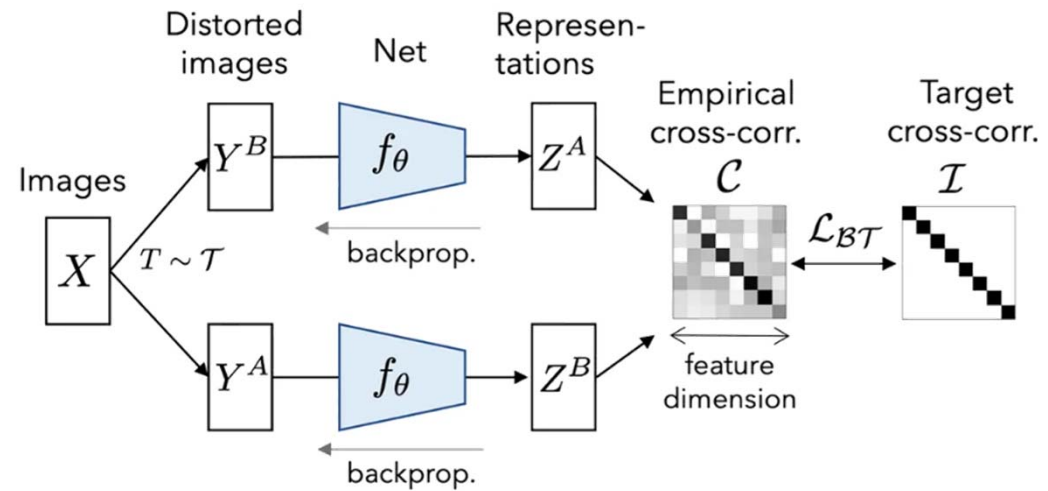
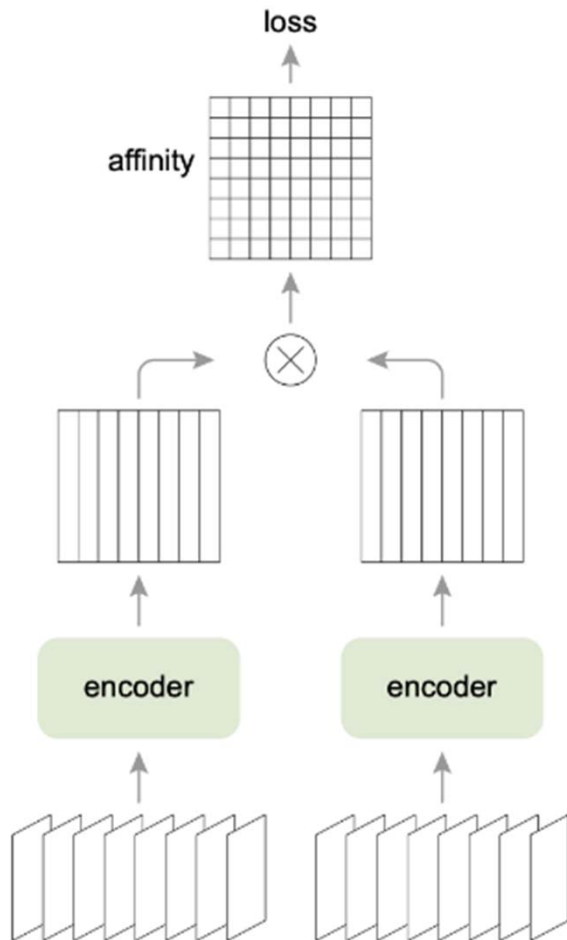
https://music-classification.github.io/tutorial/part5_beyond/methods.html

- “Draw closer” positive data pairs while “push away” negative data pairs
 - **positive pairs:** different “augmented views” of the same instance (done by *data augmentation*)
 - **negative pairs:** different instances
- **Metric learning**
 - shared encoder (CNN+MLP), output an *embedding vector*
 - learn a similarity metric discriminatively



Ref: Chen et al., “A simple framework for contrastive learning of visual representations,” ICML 2020

SimCLR



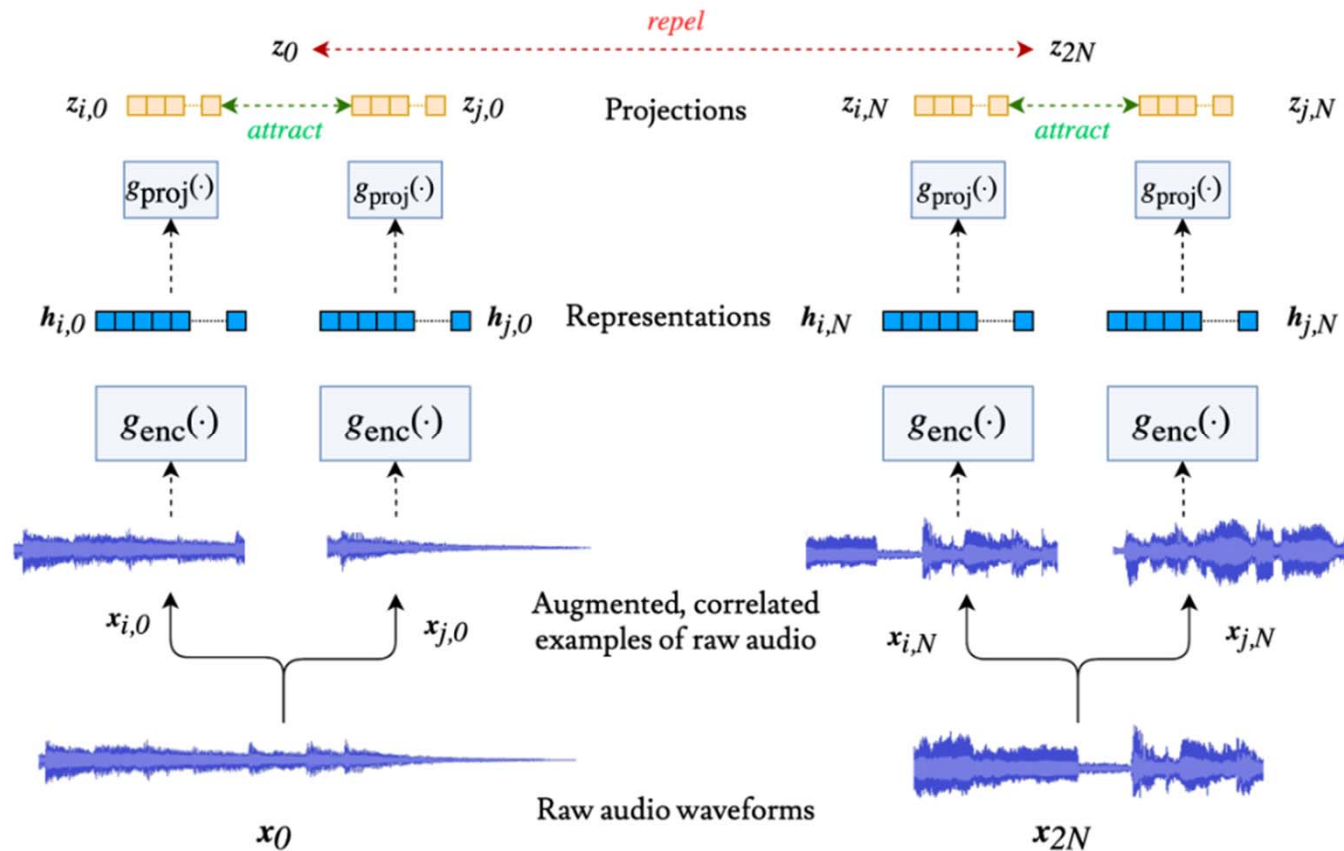
- Training details

- given a mini-batch of N samples, we will have $2N$ samples after data augmentation
- given a positive pair, view the other $2N-1$ pairs as negative
- infoNCE loss

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

Exemplar Model: CLMR

https://music-classification.github.io/tutorial/part5_beyond/self-supervised-learning.html



Ref: Spijkervet & Burgoyne, "Contrastive Learning of Musical Representations," ISMIR 2021

Sample Code

https://music-classification.github.io/tutorial/part5_beyond/self-supervised-learning.html

- Use **sample-level** CNN as the feature extractor
 - 59,049 (3^{10}) samples at 22kHz (~2.68 seconds)
- Use SimCLR for learning
- Freeze the pre-trained feature extractor, **fine-tune *linear classifier***
 - supervised SampleCNN: 49.6%
 - SSL CLMR + SampleCNN: 55.2%

Sample Code: Creating Positive Pairs

https://music-classification.github.io/tutorial/part5_beyond/self-supervised-learning.html

```
from torchaudio_augmentations import (
    Compose,
    Delay,
    Gain,
    HighLowPass,
    Noise,
    PitchShift,
    PolarityInversion,
    RandomApply,
    RandomResizedCrop,
    Reverb,
)
```

```
train_loader = get_dataloader(
    data_path="../../codes/split",
    split="train",
    is_augmentation=True,
    num_samples=59049,
    batch_size=args.batch_size,
)
iter_train_loader = iter(train_loader)
(train_wav_i, _), train_genre = next(iter_train_loader)
```

```
args.transforms_polarity = 0.8
args.transforms_filters = 0.6
args.transforms_noise = 0.1
args.transforms_gain = 0.3
args.transforms_delay = 0.4
args.transforms_pitch = 0.4
args.transforms_reverb = 0.4

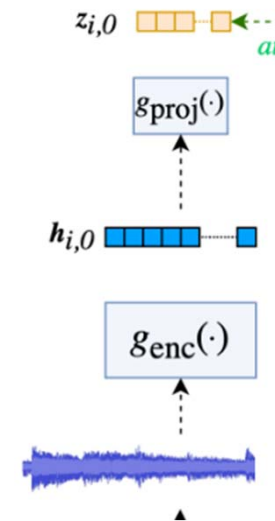
|
train_transform = [
    RandomResizedCrop(n_samples=args.audio_length),
    RandomApply([PolarityInversion()], p=args.transforms_polarity),
    RandomApply([Noise()], p=args.transforms_noise),
    RandomApply([Gain()], p=args.transforms_gain),
    RandomApply([HighLowPass(sample_rate=args.sample_rate)], p=args.transforms_filters),
    RandomApply([Delay(sample_rate=args.sample_rate)], p=args.transforms_delay),
    RandomApply([PitchShift(n_samples=args.audio_length, sample_rate=args.sample_rate)], p=args.transforms_pitch),
    RandomApply([Reverb(sample_rate=args.sample_rate)], p=args.transforms_reverb),
]
train_loader.augmentation = Compose(train_transform)
```

Sample Code: SampleCNN Encoder

- 59,049 raw audio samples (2.678 sec) as input
- Small filter size (kernel size= 3) and strides
- 1D conv + max-pooling (with factor-3 reductions)
- Finally, a **512-D vector** before sigmoid
- We use the 512-D vector for contrastive learning
 - ***h***: SampleCNN encoder output
 - ***z***: Output of the projection layer

```
# Replace the fc layer with an Identity function
self.encoder.fc = Identity()

# We use a MLP with one hidden layer to obtain  $z_i = g(h_i) = W(2)\sigma(W$ 
self.projector = nn.Sequential(
    nn.Linear(self.n_features, self.n_features, bias=False),
    nn.ReLU(),
    nn.Linear(self.n_features, projection_dim, bias=False),
)
```



3 ⁹ model, 19683 frames 59049 samples (2678 ms) as input			
layer	stride	output	# of params
conv 3-128	3	19683×128	512
conv 3-128	1	19683×128	49280
maxpool 3	3	6561×128	
conv 3-128	1	6561×128	49280
maxpool 3	3	2187×128	
conv 3-256	1	2187×256	98560
maxpool 3	3	729×256	
conv 3-256	1	729×256	196864
maxpool 3	3	243×256	
conv 3-256	1	243×256	196864
maxpool 3	3	81×256	
conv 3-256	1	81×256	196864
maxpool 3	3	27×256	
conv 3-256	1	27×256	196864
maxpool 3	3	9×256	
conv 3-256	1	9×256	196864
maxpool 3	3	3×256	
conv 3-512	1	3×512	393728
maxpool 3	3	1×512	
conv 1-512	1	1×512	262656
dropout 0.5	—	1×512	
sigmoid	—	50	25650
Total params			1.9×10^6

Sample Code: InfoNCE



- $z_{i,0}$ and $z_{j,0}$ are both from input 0, so they form a positive pair
- Similarly, $z_{i,N}$ and $z_{j,N}$ form a positive pair
- Originally, the positive pairs are located N columns away, off-diagonal
- With some code trick, the positive pairs align directly along the **main diagonal**

Nominator																		
	zi1	zi2	zi3	zi4	zj1	zj2	zj3	zj4			zj1	zj2	zj3	zj4	zi1	zi2	zi3	zi4
zi1											zi1							
zi2											zi2							
zi3											zi3							
zi4											zi4							
zj1											zj1							
zj2											zj2							
zj3											zj3							
zj4											zj4							

</

$$\exp(\text{sim}(z_i, z_j)/\tau)$$

positive samples on diagonal

dim: (batch_size * 2) x 1

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}$$

Sample Code: InfoNCE (Cont')

- The denominator acts like a Softmax normalizer; it sums the exponentiated similarities across all pairs for a given sample, excluding self-similarity
 - Self-similarity (blue): masked out ($\mathbb{1}_{[k \neq i]}$)
 - Positive pair (red)
 - Negative pair (white): All other $2N-2$ entries in that row are from different audio clips

Denominator								
	zi1	zi2	zi3	zi4	zj1	zj2	zj3	zj4
zi1								
zi2								
zi3								
zi4								
zj1								
zj2								
zj3								
zj4								

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

$$= \frac{\text{Similarity with Positive Pair}}{\text{Sum of Similarities with All Available Options}}$$

Sample Code: Supervised Finetuning

https://music-classification.github.io/tutorial/part5_beyond/self-supervised-learning.html

```
def train_linear_model(encoder, linear_model, epochs, learning_rate):

    # we now use a regular CrossEntropy Loss to compare our predicted genre labels
    criterion = nn.CrossEntropyLoss()

    losses = []
    for e in range(epochs):
        epoch_losses = []
        for (x, _) in tqdm(train_loader):

            optimizer.zero_grad()

            # we will not be backpropagating the gradients of the SampleCNN encoder
            with torch.no_grad():
                h = encoder(x)

            p = linear_model(h)

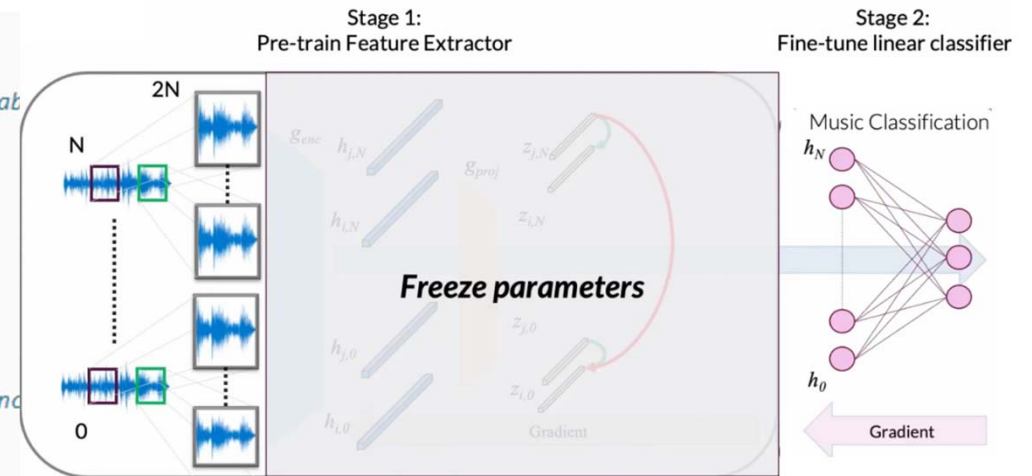
            loss = criterion(p, y)

            loss.backward()
            optimizer.step()

            # print(f"Loss: {loss}")
            epoch_losses.append(loss.detach().item())

        mean_loss = np.array(epoch_losses).mean()
        losses.append(mean_loss)
        print(f"Epoch: {e}\tMean loss: {mean_loss}")
    return losses
```

Use h (not z) for finetuning



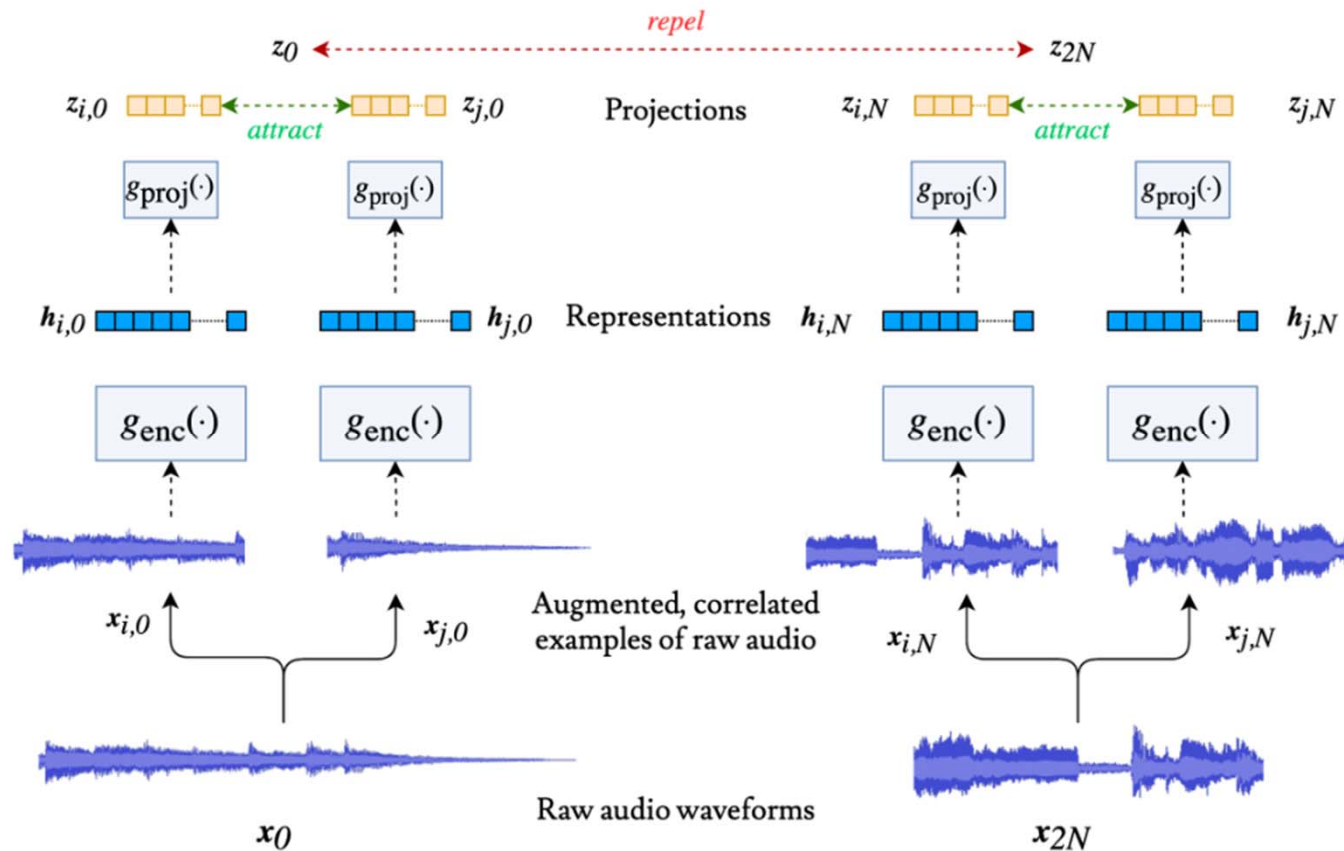
```
class LinearModel(nn.Module):
    def __init__(self, hidden_dim, output_dim):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.output_dim = output_dim
        self.model = nn.Linear(self.hidden_dim, self.output_dim)

    def forward(self, x):
        return self.model(x)
```

```
linear_model = LinearModel(n_features, args.n_classes)
```

Exemplar Model: CLMR

https://music-classification.github.io/tutorial/part5_beyond/self-supervised-learning.html



Ref: Spijkervet & Burgoyne, "Contrastive Learning of Musical Representations," ISMIR 2021

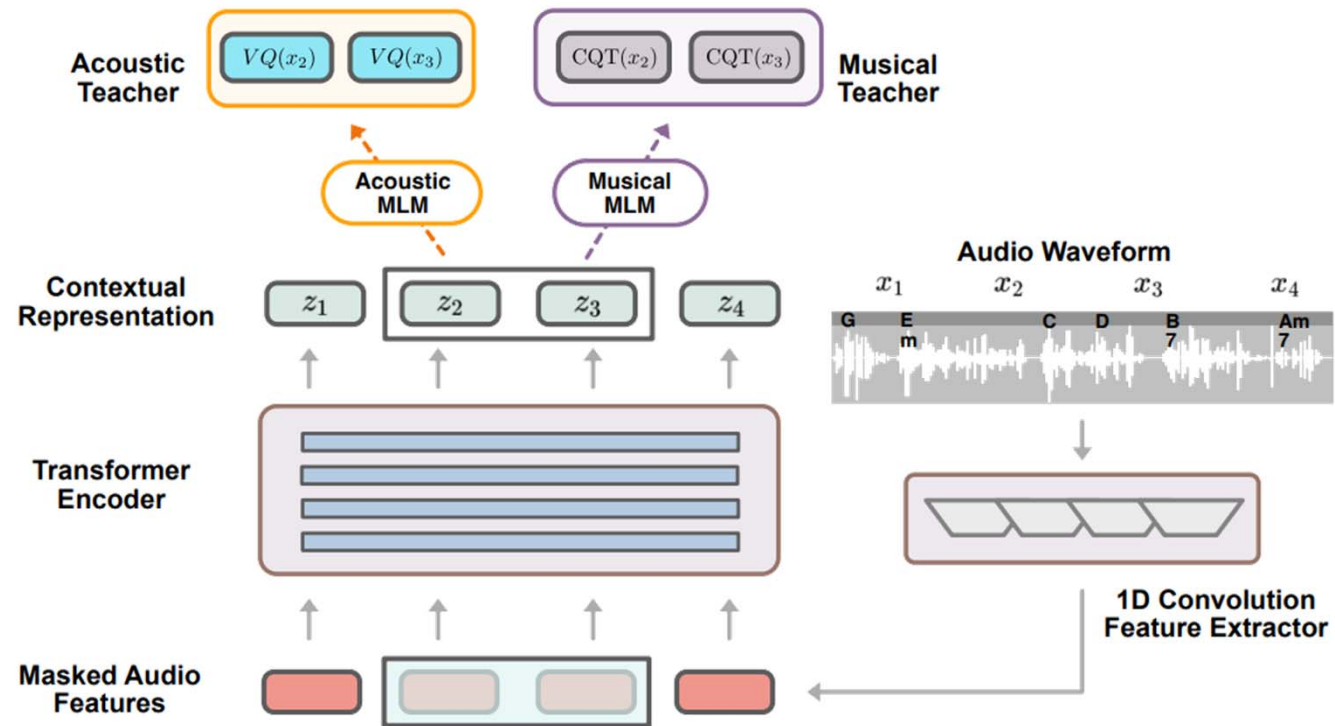
Pre-Training Strategies

- “Foundation models are pre-trained in a self-supervised fashion on large-scale datasets, avoiding or minimising the need for labelled data”
- Mainstream strategies
 - Generative pre-training (e.g., AE or VQVAE)
 - **Contrastive learning (e.g., SimCLR)**
 - **Masked modelling (e.g., BERT or MAE)**
 - Predictive joint-embedding (e.g., JEPA)

MERT: “M”usic Version of BERT

<https://github.com/yizhilll/MERT>

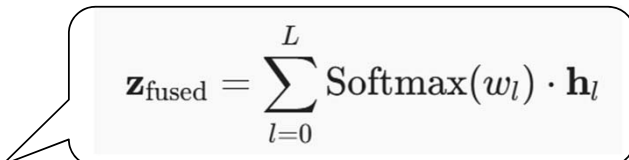
- Use **sample-level CNNs**
 - **5-second** clips as input
 - 75 embeddings per second
- Two sizes
 - 95M: 12 Transformer layers
 - 330M: 24 Transformer layers
- **Masked language modeling**
 - acoustic teacher ($\mathcal{L}_H$)
 - musical teacher ($\mathcal{L}_{CQT}$)



Ref: Li et al., “MERT: Acoustic music understanding model with large-scale self-supervised training,” ICLR 2024

MERT

- Why two teachers?
 - **acoustic teacher** ($\mathcal{L}_H$): focus on timbre via predicting HuBERT or Encodec embeddings
 - **musical teacher** ($\mathcal{L}_{CQT}$): focus on pitch/harmonic information via predicting CQT
 - adding the music teacher led to direct improvements in pitch classification, chord recognition, key detection, and beat tracking tasks
 - middle Transformer layers tend to capture pitch-related harmonic properties, while deeper layers integrate complex temporal and global acoustic context
- **Finetuning strategy**
 - freeze the backbone pre-trained models
 - get a learnable **weight sum of the output of all Transformer layers**
 - train a simple one linear layer (“**linear probe**”) for downstream frame-level tasks
 - do **pooling** to get an aggregated vector if the downstream task is clip-level


$$\mathbf{z}_{\text{fused}} = \sum_{l=0}^L \text{Softmax}(w_l) \cdot \mathbf{h}_l$$

Ref: Li et al., “MERT: Acoustic music understanding model with large-scale self-supervised training,” ICLR 2024

MARBLE Benchmark

Music Audio Representation Benchmark for universal Evaluation

<https://marble-bm.shef.ac.uk/>

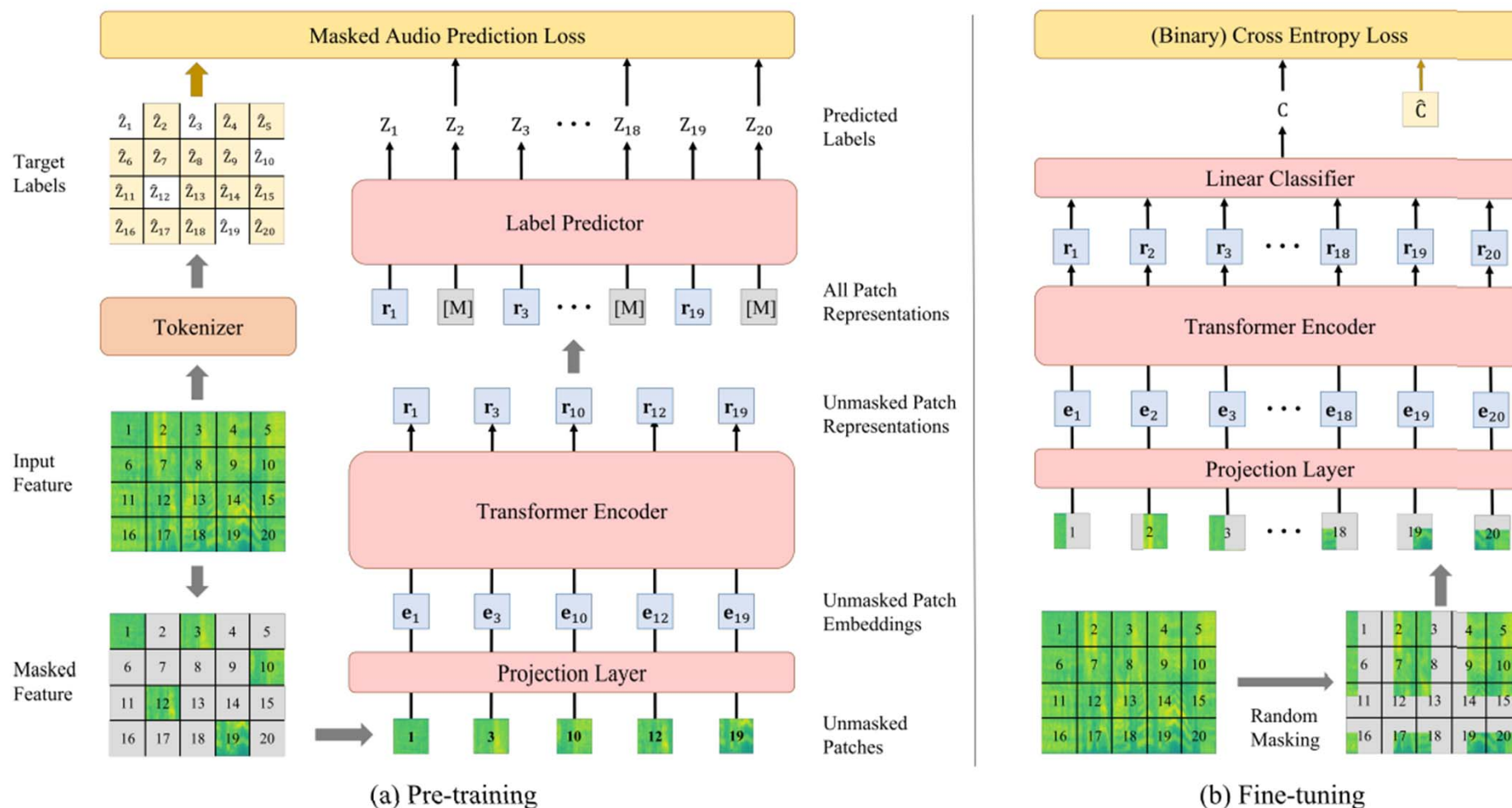
Dataset	MTT		GS	GTZAN	GTZAN	EMO		Nsynth	NSynth	VocalSet	VocalSet
Task	Tagging		Key	Genre	Rhythm	Emotion		Instrument	Pitch	Tech	Singer
Metric	ROC	AP	Acc ^{Refined}	Acc	F1 ^{beat}	R2 ^V	R2 ^A	Acc	Acc	Acc	Acc
MAP-MERT-v0-95M	90.7	38.2	64.1	74.8	88.3	52.9	69.9	70.4	92.3	73.6	77.0
MAP-MERT-v0-95M-public	90.7	38.4	67.3	72.8	88.1	59.1	72.8	70.4	92.3	75.6	78.0
MAP-MERT-v1-95M	91.0	39.3	63.5	74.8	88.3	55.5	76.3	70.7	92.6	74.2	83.7
MAP-MERT-v1-330M	91.1	39.5	61.7	77.6	87.9	59.0	75.8	72.6	94.4	76.9	87.1
MAP-Music2Vec	90.0	36.2	50.6	74.1	68.2	52.1	71.0	69.3	93.1	71.1	81.4
MusiCNN	90.3	37.8	14.4	73.5	-	44.0	68.8	72.6	64.1	70.3	57.0
CLMR	89.5	36.0	14.8	65.2	-	44.4	70.3	67.9	47.0	58.1	49.9
Jukebox-5B	91.4	40.6	63.8	77.9	-	57.0	73.0	70.4	91.6	76.7	82.6
MULE	91.2	40.1	64.9	75.5	-	60.7	73.1	74.6	88.5	75.5	87.5

MERT Outperforms CLMR

<https://marble-bm.shef.ac.uk/>

Dataset	MTT		GS	GTZAN	GTZAN	EMO		Nsynth	NSynth	VocalSet	VocalSet
Task	Tagging		Key	Genre	Rhythm	Emotion		Instrument	Pitch	Tech	Singer
Metric	ROC	AP	Acc ^{Refined}	Acc	F1 ^{beat}	R2 ^V	R2 ^A	Acc	Acc	Acc	Acc
MAP-MERT-v0-95M	90.7	38.2	64.1	74.8	<u>88.3</u>	52.9	69.9	70.4	92.3	73.6	77.0
MAP-MERT-v0-95M-public	90.7	38.4	<u>67.3</u>	72.8	88.1	59.1	72.8	70.4	92.3	75.6	78.0
MAP-MERT-v1-95M	91.0	39.3	63.5	74.8	<u>88.3</u>	55.5	<u>76.3</u>	70.7	92.6	74.2	83.7
MAP-MERT-v1-330M	91.1	39.5	61.7	77.6	87.9	59.0	75.8	72.6	<u>94.4</u>	<u>76.9</u>	87.1
MAP-Music2Vec	90.0	36.2	50.6	74.1	68.2	52.1	71.0	69.3	93.1	71.1	81.4
MusiCNN	90.3	37.8	14.4	73.5	-	44.0	68.8	72.6	64.1	70.3	57.0
CLMR	89.5	36.0	14.8	65.2	-	44.4	70.3	67.9	47.0	58.1	49.9
Jukebox-5B	<u>91.4</u>	<u>40.6</u>	63.8	<u>77.9</u>	-	57.0	73.0	70.4	91.6	76.7	82.6
MULE	91.2	40.1	64.9	75.5	-	<u>60.7</u>	73.1	<u>74.6</u>	88.5	75.5	<u>87.5</u>

BEATS: Mel-Spectrogram + Acoustic Teacher Only

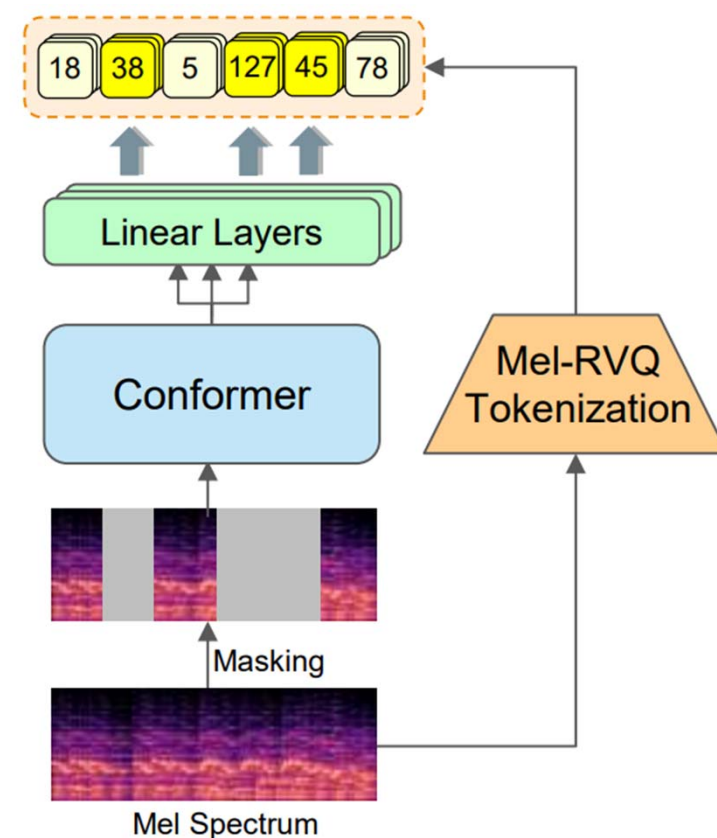


Ref: Chen et al., "BEATs: Audio Pre-Training with Acoustic Tokenizers," ICML 2023

MuQ: Mel-Spectrogram + Acoustic Teacher Only

<https://github.com/tencent-ailab/muq>

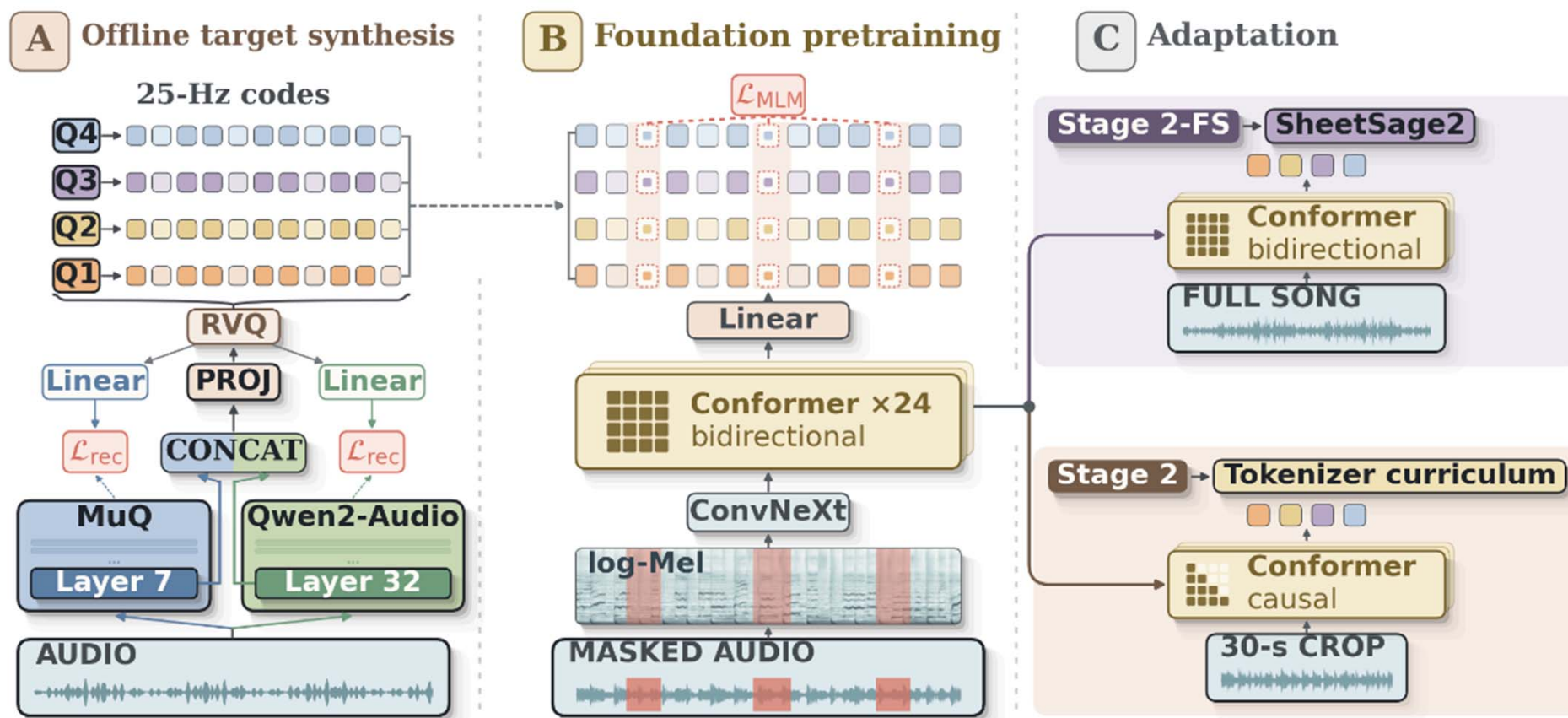
- Outperforms previous self-supervised music representation models with only 0.9K hours of open-source pre-training data
 - All pre-training experiments are running with 32 A100- 40G GPUs
 - Scaling up the data to over 160K hours and adopting iterative training consistently improve the model performance



Ref: Zhu et al., “MuQ: Self-supervised music representation learning with mel residual vector quantization,” TASLP 2025

MERT-v2

<https://huggingface.co/m-a-p/MERT-v2-30s>



Model	Params	MTT ROC	MTT AP	Key acc.	Genre acc.	Beat F1	Valence R ²	Arousal R ²
MERT-Large	330M	90.6	37.9	64.1	77.6	86.8	56.7	76.1
Dasheng-1.2B	1.2B	91.5	40.4	58.0	81.4	87.7	57.4	75.0
MuQ	310M	90.5	38.5	63.2	83.8	90.1	58.3	76.4
MusicFM	330M	90.9	38.3	63.0	84.1	90.2	57.2	74.4
AudioMAE++	307M	91.2	39.5	61.7	80.3	90.0	59.0	75.7
MATPAC++	307M	90.6	38.2	63.7	81.4	90.1	57.8	74.7
A-JEPA	307M	91.0	39.2	65.0	83.8	90.0	57.4	74.8
PupuJEPA- Large	307M	91.7	40.8	66.1	86.9	91.0	62.5	76.8
PupuJEPA- Huge	632M	91.3	39.7	64.8	85.9	90.5	62.0	78.5
MERT-v2-30s	632M	91.91	41.29	66.97	91.72	90.59	63.23	80.01
MERT-v2- FullSong	632M	91.74	41.20	67.05	90.69	90.57	63.52	78.14

MERT-v2

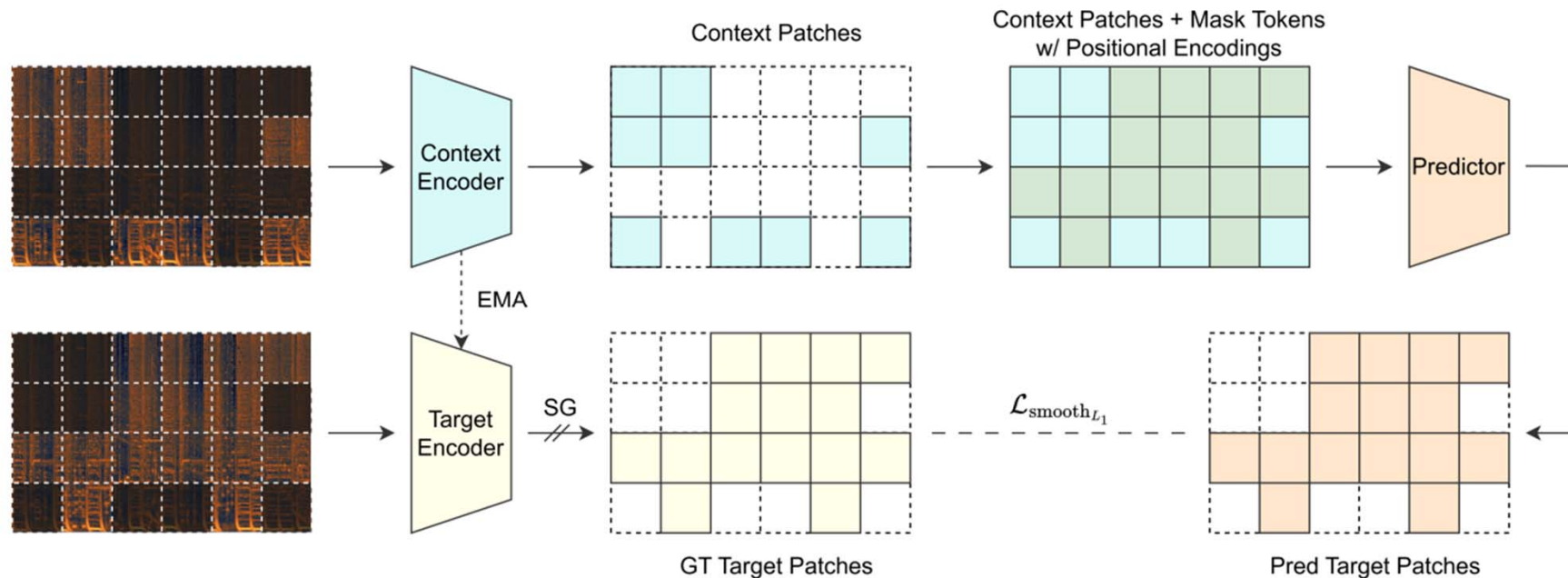
- Layer guide (“All-layer MLP” uses all 24 layers)

Task / dataset	MERT-v2-30s layer	Probe LR	MERT-v2-FullSong layer	Probe LR
Genre · GTZAN	L23	5e-3	L24	5e-4
Beat · GTZAN	L21	1e-3	L23	1e-3
Key · GiantSteps	L4	1e-3	L23	1e-3
Emotion · EmoMusic	All-layer MLP	5e-5	L24	5e-4
Chords · Chords1217	All-layer MLP	1e-4	All-layer MLP	5e-4
Tagging · MagnaTagATune	L22	1e-3	L23	1e-3
Instrument · MTG- Jamendo	L14	1e-3	L12	1e-3
Mood/theme · MTG- Jamendo	L16	1e-3	L13	1e-3
Genre · MTG-Jamendo	L19	1e-3	L16	1e-3
Top-50 · MTG-Jamendo	L13	1e-3	L22	1e-3

PupuJEPA

<https://github.com/sizigi/PupuM2D>

- JEPA: joint embedding predictive architecture (predict missing latents)

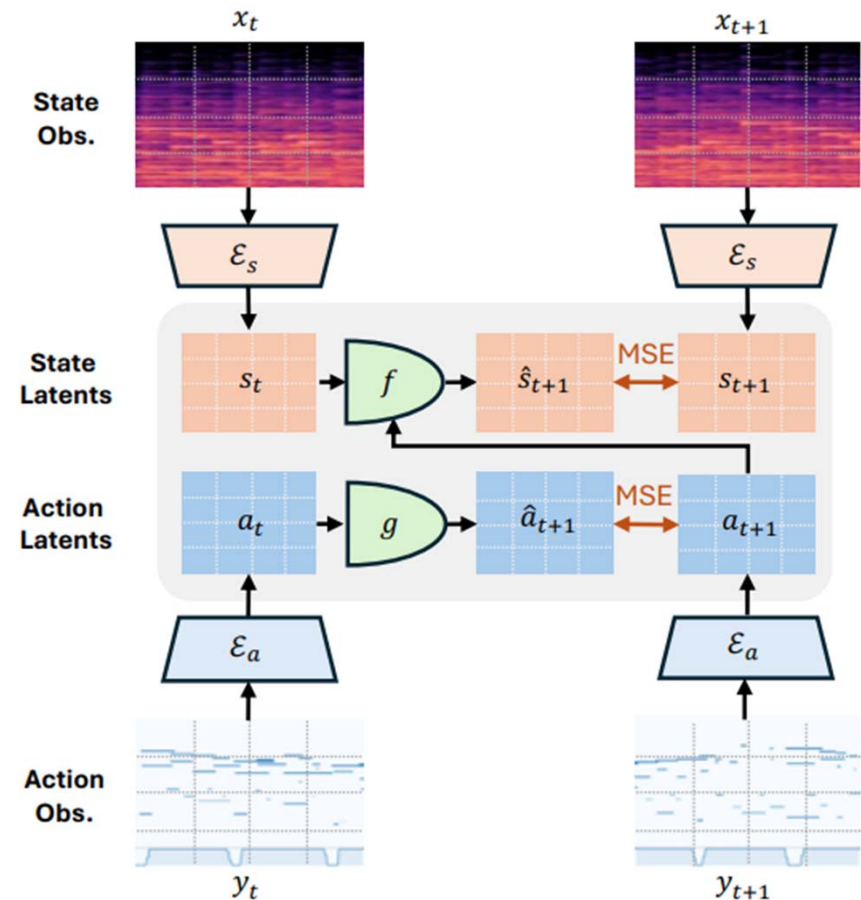


Music-JEPA

<https://zzwaang.github.io/music-jepa-demo/>

- A world model of piano sound using JEPA by framing music as an action-conditioned system
 - The audio is treated as the state, and pianoroll as the instrument action
 - Given a current audio state and an action, the model predicts the resulting future audio state

Ref: Wang et al., “Music-JEPA: Learning a world model of sound from action,” arXiv 2026



Summary

- Many pre-training strategies
 - Predict/reconstruct in **raw data space**
 - Generative pre-training (e.g., AE or VQVAE)
 - Masked modelling (e.g., BERT or MAE)
 - Operate entirely in **latent representation space**
 - Contrastive learning (e.g., SimCLR)
 - Predictive joint-embedding (e.g., JEPA)
- Finetuning via linear probing
 - Take a look at MARBLE: <https://github.com/a43992899/MARBLE>
 - Marble is a modular, configuration-driven suite for training, evaluating, and **performing inference on state-of-the-art pretrained music models**

Encoders

Name	Description	Paper	Link
CLaMP3	Cross-Modal & Language-based Music Pretraining v3. Aligns audio, sheet music, MIDI, and multilingual text via contrastive learning.	arXiv:2502.10362	GitHub sanderwood/clamp3
DaSheng	Deep Audio-Signal Holistic Embeddings: masked autoencoder trained on 272 k h of diverse audio.	arXiv:2406.06992	GitHub richermans/dasheng
identity	Pass-through encoder.	—	—
MERT	Music understanding via large-scale self-supervised training with acoustic & musical pseudo-labels.	arXiv:2306.00107	GitHub yizhilll/MERT
MuQ	Self-supervised music representation with Mel Residual Vector Quantization.	arXiv:2501.01108	GitHub Tencent-ailab/MuQ
MuQMulan	Two-tower contrastive model combining MuQ audio and text for zero-shot tagging.	arXiv:2501.01108	Hugging Face OpenMuQ/MuQ-MuLan-large
MusicFM	Masked-token modeling in music using random projections & codebooks.	arXiv:2311.03318	GitHub minzwon/musicfm
Qwen2_5OmniEncoder	Qwen 2.5-Omni Audio Tower: a multimodal generalist model supporting text, image, audio, and video.	arXiv:2503.20215	GitHub QwenLM/Qwen2.5-Omni